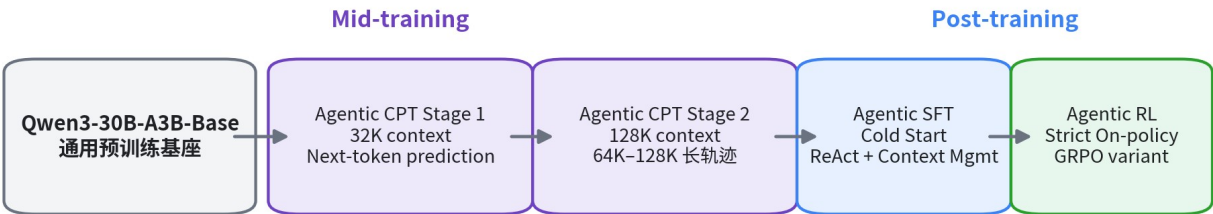


Tongyi DeepResearch

完整技术研究：从 Synthetic Data 到 Agentic CPT / SFT / RL

模型 · 数据 · 环境 · 训练 · 推理 · 源码 · 评测 · Know-why / Know-how

端到端 Agent Training Pipeline



Know-why：先用 CPT 建立 agentic inductive bias，再用 SFT 建立稳定先验，最后用 RL 学会探索与策略优化。

研究日期：2026-08-13

主要依据：Tongyi DeepResearch Technical Report v3（2026-05-18）、Alibaba-NLP/DeepResearch 官方仓库、Tongyi Agent 官方技术博客、Hugging Face 官方模型卡。
研究方法：官方事实 / 源码观察 / 工程推断分层呈现。

目录

1. 执行摘要：Tongyi DeepResearch 为什么重要
2. 定位与技术演化：从 Web Agent 到 Agentic Model
3. 模型与 Agent 形式化：30.5B MoE + 工具交互
4. 设计原则：Agentic Mid-training + Post-training
5. Synthetic Data-Centric Scaling
6. Environment Scaling：Prior / Simulation / Real
7. Agentic CPT：32K → 128K 的中训练桥梁
8. Post-training Data：Graph → Subgraph → Uncertainty
9. SFT Cold Start：ReAct + Context Management
10. Agentic RL：严格 On-policy 的 GRPO 变体
11. Automatic Data Curation 与 Policy-Data 共演化
12. RL 基础设施：异步 Rollout + Unified Sandbox
13. 推理：Native ReAct 与 Heavy / IterResearch
14. 开源源码：Inference Loop 与 Tool Stack
15. Benchmark 结果、实验设置与可信度边界
16. 实验 Know-how：Context / Interaction / Simulation
17. AgentFounder / AgentScaler 与 Tongyi Agent 家族
18. 与 GPT Researcher / LangChain ODR / DeerFlow 对比
19. 可复用 Know-how：如何训练自己的 Deep Research Agent
20. 复现、部署、局限与建议
21. 关键结论与参考来源

1. 执行摘要：Tongyi DeepResearch 为什么重要

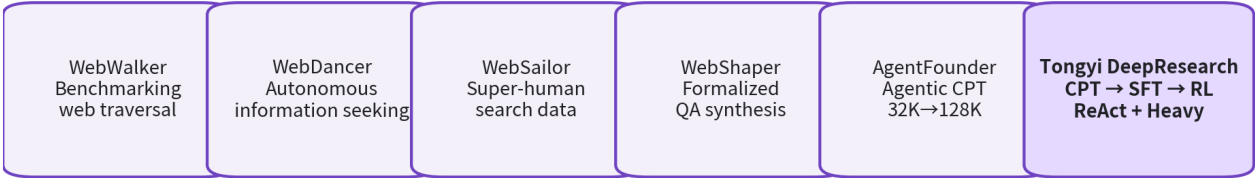
Tongyi DeepResearch 的核心价值，不是又做了一个“Search + LLM”的 Deep Research workflow，而是把 Deep Research 当成一种需要被训练出来的模型能力：让模型在长程任务中自主规划、搜索、阅读、计算、验证、压缩上下文并决定何时停止。官方技术报告把这一目标表述为“autonomous research agency”，并将训练链路从传统 post-training 向前延伸到 Agentic Mid-training。[S1]

◆ 最重要的 Know-why：通用基础模型的预训练语料大多是静态文本，缺少“研究级问题 + 多轮工具交互 + 决策轨迹”，因此只靠 SFT/RL 在最后阶段补 Agent 能力，会同时承担“学会 Agent 行为”和“做任务对齐”两种目标，容易产生优化冲突。

维度	Tongyi DeepResearch 选择	工程含义
基础模型	Qwen3-30B-A3B-Base；30.5B total / 3.3B active	MoE 用较低激活成本承载较强 agent 能力
上下文	128K	支持大量工具交互，但仍需 Context Management
训练	Agentic CPT → Agentic SFT → Agentic RL	把 Agent 能力分阶段内化
数据	全链路 synthetic-data centric	数据构造成为第一等工程对象
环境	Prior / Simulated / Real	通过环境分层控制成本、稳定性与真实性
推理	Native ReAct + Heavy/IterResearch	区分模型内生能力与 test-time scaling 上限
工具	Search / Visit / Python / Scholar / File Parser	研究任务从“文本生成”扩展为环境交互

截至 2026-08-13，官方 GitHub 仓库约 19.8k stars、1.5k forks；模型权重以 Apache-2.0 发布，官方模型卡给出 128K context。[S2][S4]

Tongyi DeepResearch 技术演化：从 Web Agent 研究到原生 Agentic Model



关键变化：从“外部 workflow 编排搜索”走向“把规划、搜索、工具使用与长程研究策略内化进模型参数”

图1 Tongyi DeepResearch 技术演化（本报告重绘）

2. 定位与技术演化：从 Web Agent 到 Agentic Model

Tongyi DeepResearch 不是孤立项目，而是 Tongyi Web Agent 系列方法的汇合点。官方仓库列出的系列工作包括 WebWalker、WebDancer、WebSailor、WebShaper、WebWatcher、WebResearcher、ReSum、WebWeaver、WebSailor-V2、AgentFounder、AgentScaler 等。[S2]

- WebWalker：把多跳网页遍历本身变成可测量的问题。
- WebDancer：研究自主信息寻求 agency。
- WebSailor / V2：重点解决超难搜索问题的数据合成与可扩展 RL。
- WebShaper：形式化 information-seeking task，使难度、结构和可验证性可控。
- AgentFounder：把 Agentic Continual Pre-training 引入 Deep Research agent。
- AgentScaler：把“环境规模”作为通用 function-calling agent 的能力来源。

因此 Tongyi DeepResearch 更像一个“研究成果集成点”：AgentFounder 提供 mid-training 的 agentic prior，WebSailor/WebShaper 提供困难问题与数据构造方法，ReSum/IterResearch 提供长程上下文管理思想，最终再通过 SFT 和 on-policy RL 形成统一模型。

◆ 演化主线不是“图越来越复杂”，而是：Benchmark → Data → Environment → Training → Context Management → Test-time Scaling。

3. 模型与 Agent 形式化：30.5B MoE + 工具交互

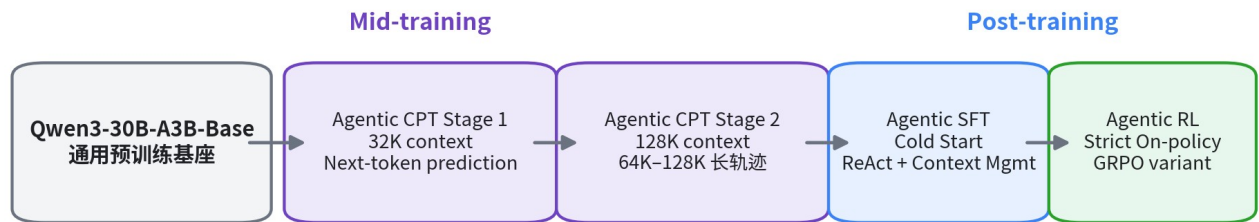
技术报告将每一步 rollout 抽象为 Thought τ_t 、Action a_t 与 Observation o_t 。这意味着训练对象并非单一答案，而是一个与环境闭环交互的 policy：给定历史状态，模型决定内部推理与工具动作，环境返回 observation，再进入下一轮。
[S1]

从开源 inference/react_agent.py 可以看到一个直接的 MultiTurn ReAct loop：模型最多进行由 MAX_LLM_CALL_PER_RUN 控制的多轮调用（默认环境变量未设置时为 100）；每轮解析 <tool_call>，执行工具并将 <tool_response> 写回消息；遇到 <answer> 则停止。源码还设置约 150 分钟总时限，并在 token 数超过约 110×1024 时强制要求模型停止工具调用并给出最终答案。[S7]

源码对象	作用	研究含义
MultiTurnReactAgent	维护 Thought/Tool/Observation 循环	最小 Agent Runtime
TOOL_CLASS	FileParser / Scholar / Visit / Search / PythonInterpreter	多源研究 action space
call_server	OpenAI-compatible model service + retry/backoff	推理服务与 Agent loop 解耦
count_tokens	用 tokenizer 计算上下文	硬性 Context Guard
custom_call_tool	工具分发和文件路径处理	工具抽象层

4. 设计原则：Agentic Mid-training + Post-training

端到端 Agent Training Pipeline



Know-why：先用 CPT 建立 agentic inductive bias，再用 SFT 建立稳定先验，最后用 RL 学会探索与策略优化。

图2 端到端训练链路（本报告重绘）

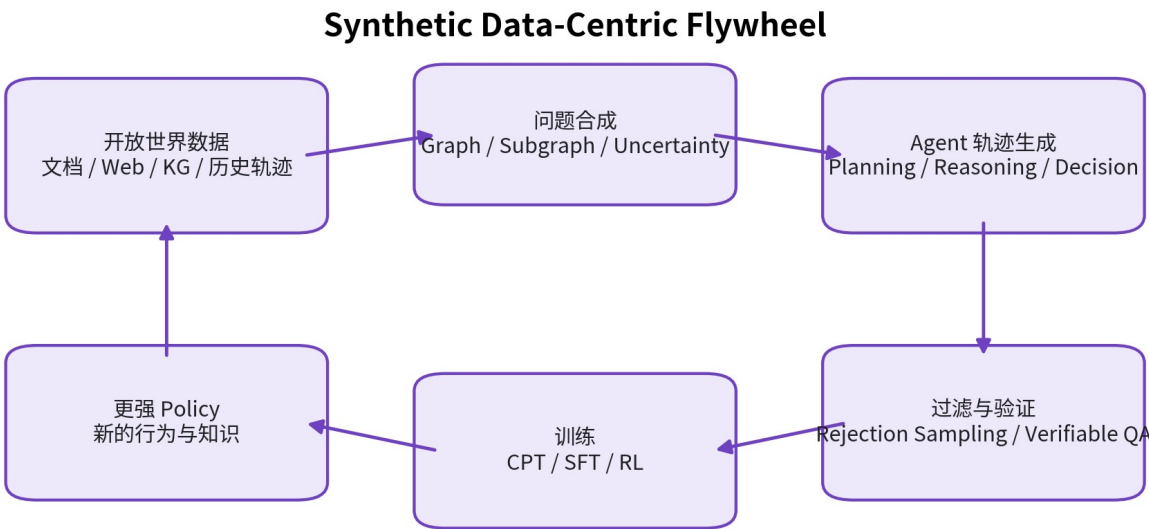
技术报告的核心判断是：一般 foundation model 缺少 agentic inductive bias。通用预训练主要来自静态文本；即使模型具备知识与推理，它也未必天然知道如何进行长程任务分解、何时搜索、如何根据 observation 改变计划、如何维持多轮目标。[S1]

因此训练被拆成两个宏阶段：

1. Mid-training：用大规模 agentic behavior data 将模型从“文本模型”推向“Agent foundation model”，目标是形成工具使用、规划、决策和长轨迹的先验。
2. Post-training：先用 SFT cold start 获得稳定格式与基本 research workflow，再通过 RL 与环境闭环，探索更优搜索和决策策略。

这种分阶段设计把“模仿”和“探索”分开。SFT 擅长行为克隆、格式对齐和稳定启动，但很难自己发现更优策略；RL 则利用可验证 reward 反过来优化策略，并能发现 demonstration 中不存在的行为。

5. Synthetic Data-Centric Scaling



核心不是“合成更多数据”，而是让数据难度、轨迹质量与当前 policy 的能力持续共演化。

图3 Synthetic Data-Centric Flywheel（本报告重绘）

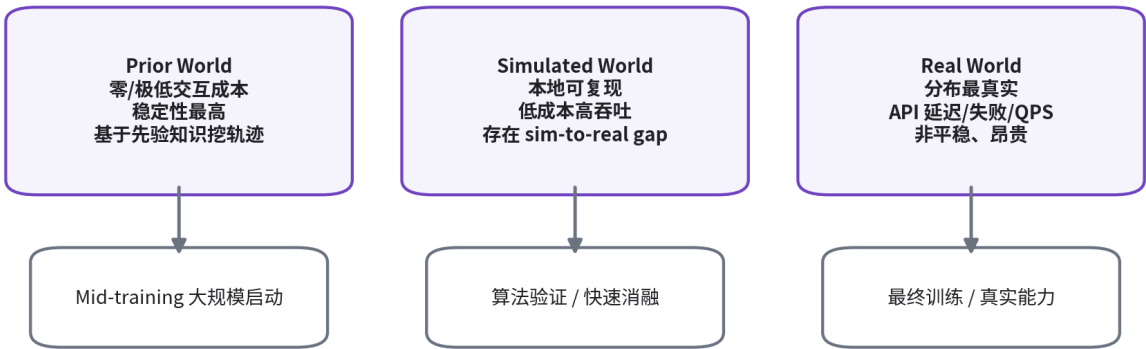
Deep Research 数据稀缺的根源是：问题必须跨来源连接信息、进行多步推理、验证结论，并伴随长工具轨迹。人工标注不仅贵，而且很难规模化保持一致。Tongyi 因此把 synthetic data 作为全训练阶段的“发动机”。[S1]

合成数据优势	为什么对 Agent 特别重要
问题规模容易扩展	LLM 能快速构造 research-level QA，而不是等待自然出现
模式与难度可控	可定向合成 planning / synthesis / memory / tool-use 等 meta-capabilities
验证相对容易	可先构造结构与答案，再生成问题，减少人工求解成本
形成数据飞轮	更强 policy 产生更好的轨迹，再反哺下一轮训练
分布更稳定	比小规模、噪声更高的人类难题集更适合持续优化

◆ **Know-how: Agent 数据不是“问答文本”这么简单，而是 Question + State + Thought + Action + Observation + Verification + Final Answer 的结构化轨迹。**

6. Environment Scaling：Prior / Simulation / Real

Environment Scaling：训练环境不是“外部世界”，而是训练系统的一部分



策略：Mid-training 以 Prior/Simulation 为主；Post-training 先在 Simulation 验证，再进入 Real World。

图4 三层环境设计（本报告重绘）

Tongyi 的另一个关键重构是：环境并非被动的“真实世界 API”，而应该被主动设计为训练系统的一部分。[S1]

- **Prior World**：提供任务元素、工具与状态定义，agent 基于预训练知识挖掘交互轨迹；稳定、近零调用成本，但缺乏真实反馈。
- **Simulated Environment**：在本地构造受控、可复现的真实交互副本；低成本、高吞吐、便于因果分析，但存在 sim-to-real gap。
- **Real-world Environment**：分布最真实，是最终能力试金石；同时也是最贵、最非平稳、最容易因 API 错误污染训练的环境。

官方策略是：mid-training 主要利用 Prior World 与 Simulation 做大规模 bootstrap；post-training 先在 simulation 中验证训练策略，再把验证过的策略迁移到真实环境。[S1]

7. Agentic CPT：32K → 128K 的中训练桥梁

Tongyi DeepResearch 采用两阶段 Agentic Continual Pre-training（Agentic CPT）。Stage 1 使用 32K context，Stage 2 扩展到 128K，并在第二阶段显著加入 64K–128K 的长 agent trajectory。优化仍然采用标准 next-token prediction loss。[S1]

这一步的目标不是奖励优化，而是“让模型见过足够多的 Agent 世界”。它仍然像预训练一样做 next-token prediction，但 token 序列里不再只是文章，而包含问题、计划、搜索动作、观察、判断与答案。

CPT 数据类型	训练的隐含能力
Multi-style Question Synthesis	认识不同 research task 的结构
Planning Action	问题分解、第一步动作预测
Reasoning Action	从异构证据中蒸馏关键知识并推理
Decision-Making Action	在多个候选 action path 中做选择
General Function Calling	跨异构模拟环境迁移工具调用能力

AgentFounder 官方 README 进一步强调 FAS（planning/reasoning action synthesis）与 HAS（decision-making action synthesis），并报告随着训练数据扩大，多个 benchmark 的平均表现持续提升，呈现潜在 scaling-law 特征。[S5]

8. Post-training Data：Graph → Subgraph → Uncertainty

Post-training 数据需要两个额外属性：足够难，并且可验证。Tongyi 的高质量 QA 合成链路从 Web 随机游走构建互联知识图谱，并引入真实网页表格；随后采样 subgraph/subtable 生成初始 QA，再通过 uncertainty injection 主动增加难度。[S1][S3]

阶段	操作	目的
Graph Construction	Web random walk + 表格融合	建立真实信息结构
Subgraph Sampling	抽取需要多源连接的局部结构	构造多跳 reasoning dependency
Uncertainty Injection	实体模糊、属性合并等原子操作	提高检索不确定性并避免直达答案
Formalization	集合论描述 information-seeking	控制难度和结构、减少 reasoning shortcut
Verification	利用已知结构验证 QA	使 RLVR 的 0/1 reward 可落地

官方还描述了 PhD-level 问题引擎：从多学科知识库构造 seed QA，再让 question-crafting agent 使用 Web Search、Academic Retrieval、Python 等工具循环升级难度。这里的本质是“生成问题也做成 Agent task”。[S3]

9. SFT Cold Start：ReAct + Context Management

SFT 的职责是建立一个稳定的初始 policy。官方从合成高质量 QA 出发，用高性能开源模型生成包含完整 thought 与 tool response 的轨迹，再经过 rejection sampling，仅保留成功且模式多样的示例。[S1]

SFT 混合两种样本表示：

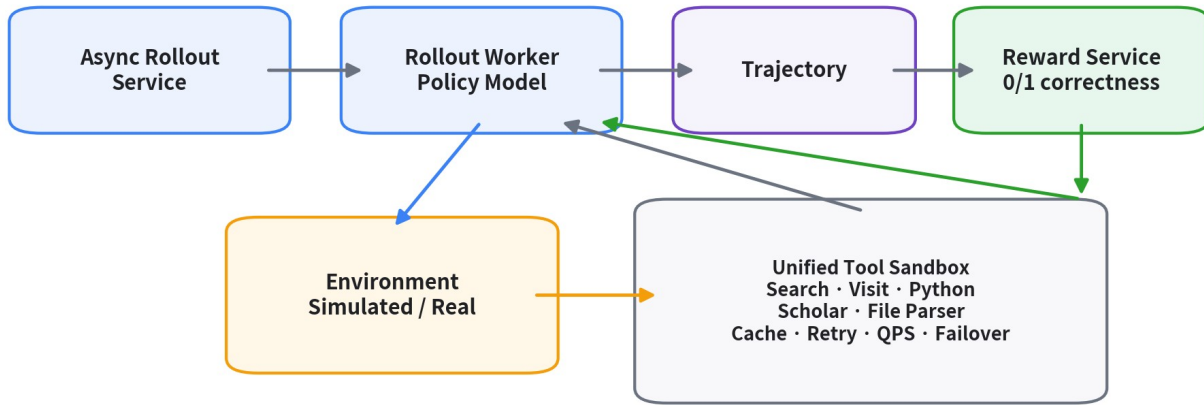
- ReAct Mode：输入历史 H_t ，输出当前 thought τ_i 与 tool call a_i 。优点是通用、简单、与多数 Agent runtime 兼容。
- Context Management Mode：输入上一步 summary $S_{(t-1)}$ 、action 与 observation，输出新的 trajectory summary、thought 与 action。它把“压缩上下文”本身训练成可学习能力。

训练同样做 context curriculum：第一阶段 40K，第二阶段扩展到 128K，并保留少量短样本以稳定训练。[S1]

◆ 这一步非常关键：Context Engineering 不只是 host 侧 middleware，也可以变成 model-side learned behavior。

10. Agentic RL：严格 On-policy 的 GRPO 变体

Agentic RL：异步 rollout + 稳定工具环境 + 动态数据课程



工程优先级：环境稳定性 / 数据质量 > 复杂算法技巧。

图5 Agentic RL 系统（本报告重绘）

Agentic RL 使用完整任务 rollout；最终答案若与 ground truth 匹配则获得 reward 1，否则 0（RLVR）。模型在 simulated 或 real environment 中持续多轮交互。[S1]

机制	Tongyi 选择	Why
Policy regime	Strictly on-policy	保证 trajectory 来自当前 policy，降低策略滞后
Base algorithm	Tailored GRPO	利用 group-relative learning，避免单独 value model
Loss granularity	Token-level policy gradient	适配长轨迹中逐 token 的 policy 更新
Exploration	clip-higher	允许更积极探索
Advantage	Leave-one-out	降低估计方差
Reward	纯 0/1 answer correctness	SFT 已负责格式，不再混入 format reward
Negative rollout	选择性过滤	避免超长未作答等负样本导致 policy/format collapse

报告明确指出，这些修改的主要动机不是追求算法新颖性，而是追求效率和训练稳定性。[S1]

可把核心目标写成：对同一个问题 q 采样 G 个 rollout，优势由该 rollout reward 相对组内平均 reward 决定；strict on-policy 时 importance ratio 近似 1。这个结构非常适合“最终答案可验证，但中间路径难人工标注”的 Deep Search 任务。

11. Automatic Data Curation 与 Policy-Data 共演化

Tongyi RL 不把训练集视为静态集合。初始 SFT policy 会对大数据集 D 的每个问题采样多个 rollout；始终失败或始终成功的问题都缺少有效学习信号，因此被过滤，留下“中等难度”集合 D' 。随着 policy 变强，已经掌握的问题被移除，同时用中间 checkpoint 从原始 D 中寻找对当前模型重新变得“适中”的新问题，刷新 active curriculum。[S1]

◆ **最可复用的 RL know-how：训练问题难度应该追着 policy 走。太容易 = 没梯度；太难 = 全零 reward；“适中难度窗口”才有最强学习信号。**

这也解释了官方强调的“data quality and environment stability > specific algorithm”。在 Agentic RL 中，reward 信号可能被搜索 API 失败、网页异常、QPS 限制、工具延迟污染；如果环境不稳定，算法再好也会学习错误因果。

12. RL 基础设施：异步 Rollout + Unified Sandbox

真实环境工具包括 Search、Visit、Python Interpreter、Google Scholar、File Parser。官方指出外部 API 的延迟、失败和返回不一致会污染轨迹，因此构建统一 sandbox / scheduling layer，实施 QPS 控制、缓存、timeout+retry、非关键失败降级和备用数据源 failover。[S1]

Rollout 本身采用 step-level asynchronous RL loop，基于 rLLM 扩展：模型推理服务与工具调用服务分离，通过统一 interaction handler 合并反馈；多个 agent instance 可并发与环境交互并独立完成 rollout。[S1][S3]

工程问题	解决手段
Search API QPS	rate limit + concurrency control
网页/工具超时	timeout + retry
数据源故障	backup provider / failover
重复访问成本	result cache
训练吞吐受 tool latency 限制	model server 与 tool server 异步分离
错误原因不可诊断	统一接口，减少环境随机性

13. 推理：Native ReAct 与 Heavy / IterResearch

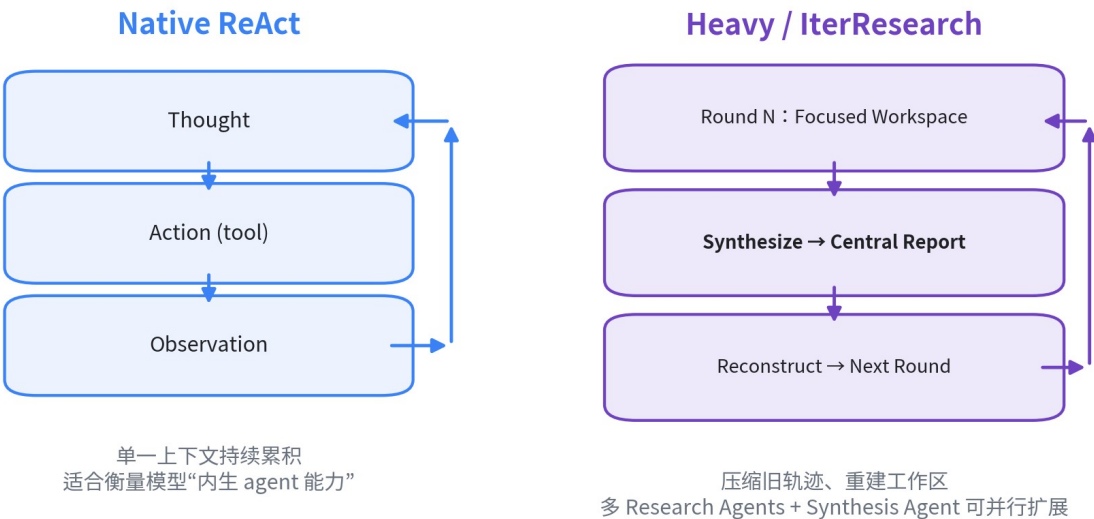


图6 两种推理范式（本报告重绘）

Native ReAct 是评测“模型内生能力”的基线：不依赖复杂 prompt engineering，严格遵循 Thought → Action → Observation，多轮直到答案。官方将其与“Bitter Lesson”联系起来：优先验证能够随模型和计算扩展的简单通用方法。[S3]

Heavy Mode 则面向极长研究任务。IterResearch 不是把所有 observation 永久堆积在一个 context，而是把任务分成 research rounds；每轮只继承上一轮最重要的结果，重新构造 focused workspace，并持续更新 central report。这样把 “synthesis + reconstruction” 作为长程工作记忆机制。[S3]

在更高层的 Research-Synthesis 中，多个 Research Agents 并行执行 IterResearch，最终由 Synthesis Agent 汇总精炼报告。它本质上将 test-time scaling 从 “多吐 token” 扩展为 “更多环境 interaction + 多条独立研究路径”。

14. 开源源码：Inference Loop 与 Tool Stack

官方仓库将 inference、evaluation、Agent 系列研究拆开。公开复现的核心 inference 目录包含 react_agent.py、run_multi_react.py、prompt.py，以及 file/search/visit/scholar/python 五类工具实现。[S2][S7]

文件	职责	值得借鉴
react_agent.py	Multi-turn ReAct 主循环	最小可复现 Agent runtime + token/time/call guard
tool_search.py	搜索	把检索作为独立 tool provider
tool_visit.py	网页访问/阅读	Search 与 Read 分离，避免只依赖 snippet
tool_scholar.py	学术检索	按来源类型分工具
tool_python.py	Python sandbox	计算与信息检索组合
tool_file.py	文件解析	支持 document research
evaluation/*	HLE/xbench 等官方评测脚本	评测与 agent runtime 分离

源码还显示明显的生产式 guard：服务调用最多重试 10 次，指数退避并加随机 jitter；单任务有 wall-clock 上限；LLM call 有显式 budget；上下文达到阈值时强制 finalization。这些 “看起来不聪明” 的边界恰恰是 long-horizon agent 能稳定运行的前提。[S7]

15. Benchmark 结果、实验设置与可信度边界

技术报告的主要评测在 7 个 public information-seeking benchmark 上进行。统一推理参数包括 temperature=0.85、repetition penalty=1.1、top-p=0.95；每题最多 128 次工具调用、context 128K；每个 benchmark 独立跑 3 次，主指标为 Avg@3。[S1]

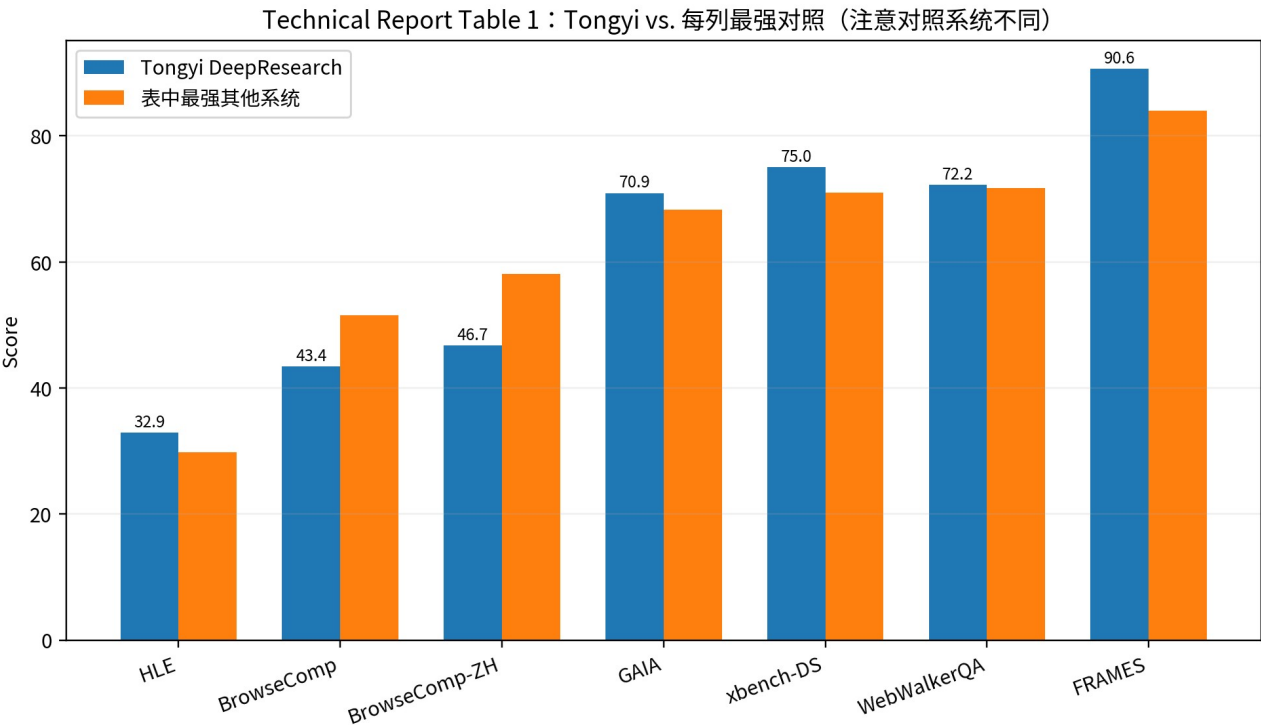


图7 报告 Table 1 的结果重绘；“其他系统”每列可能是不同模型，不能当成单一统一 baseline

Benchmark	Tongyi Avg@3	报告中的代表性对照	解释
HLE	32.9	DeepSeek-V3.1 29.8 / OpenAI DR 26.6	学术/知识+工具推理
BrowseComp	43.4	OpenAI DR 51.5 / o3 49.7	Tongyi 不是该列最高
BrowseComp-ZH	46.7	o3 58.1 / DeepSeek 49.2	中文复杂浏览
GAIA	70.9	Claude-4-Sonnet 68.3	多工具通用 Agent
xbench-DeepSearch	75.0	DeepSeek-V3.1 71.0	用户导向深搜
WebWalkerQA	72.2	o3 71.7	网页遍历
FRAMES	90.6	o3 84.0	长上下文/多证据

◆可信度边界：这些结果来自项目方技术报告；各商业系统的工具、时间预算、调用上限和版本并不完全等价。最稳妥的读法是“Tongyi 在其公开协议下非常有竞争力”，而不是把所有分数视为严格苹果对苹果的同一排行榜。

另外，报告明确标注：主要结果采集于 2025-09-16，xbench-DeepSearch-2510 于 2025-10-28；因此它们是当时快照，而不是 2026-08-13 的实时模型排名。[S1]

16. 实验 Know-how：Context / Interaction / Simulation

技术报告包含几组特别有工程价值的实验：

- 3. Reward 与 entropy：训练 reward 持续上升，entropy 保持稳定，说明动态 curriculum 能持续提供学习信号并避免过早塌缩。
- 4. RL context length：64K context 的 reward ceiling 最高；48K 次之；32K 在同一 curriculum 下会被迫学习更短、更高效的解决路径。

- 5. Interaction test-time scaling：随着 context 和 environment interaction turns 增加，BrowseComp accuracy 持续提高。Agent 的 scaling 维度不只是 reasoning tokens，还包括“可用环境观测数量”。
- 6. Super-human SFT data：报告统计超过 20% 的 SFT 样本超过 32K token 且包含超过 10 次 tool invocation，说明 cold-start 数据本身已是长程 Agent trajectory。
- 7. Simulation-to-real：基于 2024 Wikipedia 的 offline simulation 中，适配 GRPO 的 reward curve 与 real environment 走势接近，被团队比喻为“wind tunnel”式算法试验场。

◆ 非常值得迁移的实验原则：先在可复现 simulation 做算法/系统消融，再到 real world 验证；否则环境噪声会让你无法判断“policy 差”还是“工具坏”。

17. AgentFounder / AgentScaler 与 Tongyi Agent 家族

AgentFounder 是 Tongyi DeepResearch mid-training 思路的重要前序工作。官方 README 将其定义为首个把 Agentic CPT 引入 Deep Research Agent 训练 pipeline 的工作，并采用 32K/128K 两阶段持续预训练；其 action synthesis 分别针对 planning、reasoning、decision-making。[S5]

AgentScaler 则把问题向更一般的 function-calling intelligence 扩展：自动构建异构、全模拟环境，再通过两阶段 fine-tuning 学习环境交互经验。其核心命题是“environment diversity 决定 function-calling breadth”。[S6]

系列工作	主要回答的问题	沉淀到 Tongyi DR 的资产
WebWalker	如何测 Web traversal	多跳搜索 benchmark / task pattern
WebSailor	如何合成超难信息寻求数据	高难 QA 和 trajectory synthesis
WebShaper	如何形式化 information seeking	难度控制 + 可验证任务结构
AgentFounder	Agent 能否在 CPT 阶段学习	Agentic prior + 32K/128K curriculum
AgentScaler	如何扩大 function-calling 环境覆盖	environment scaling
ReSum / IterResearch	如何突破长轨迹 context 噪声	summary + workspace reconstruction

18. 与 GPT Researcher / LangChain ODR / DeerFlow 对比

Deep Research 四种路线的“工程对象”差异

系统	核心工程对象	研究策略来源	Context 管理	训练重点
GPT Researcher	Research pipeline	Workflow + prompts	检索压缩/报告聚合	基本不训练专用 agent
LangChain ODR	Graph + Supervisor	Tool-calling supervisor	Subagent 隔离 + compression	工程编排
DeerFlow 2.x	General Agent Harness	Lead Agent + Skills	Middleware + files + summary	通用 runtime
Tongyi DeepResearch	Agentic LLM 本体	CPT/SFT/RL 内化 policy	ReAct / IterResearch	Agentic data + environment + RL

图 8 四种 Deep Research 路线的工程对象（本报告归纳）

四个项目真正不同的不是“有没有 Search Tool”，而是工程优化对象不同：

- GPT Researcher：优化 research pipeline，把 query decomposition、retrieval、scrape、context compression、report writing 做成稳定工程链。
- LangChain Open Deep Research：优化 supervisor + researcher subgraph，通过并发 subagent、reflection、compression 管理复杂研究。
- DeerFlow 2.x：优化 general-purpose harness；Deep Research 更像 Skills/Tools 上的一类任务。
- Tongyi DeepResearch：优化 model policy 本身；希望模型天然知道如何搜索和计划，host 侧 runtime 相对保持简单。

因此 Tongyi 最有价值的不是拿来替换所有 Deep Research workflow，而是提供另一条路线：当任务分布稳定、训练资源足够时，把反复出现的 Agent policy 从 prompt/graph 中“蒸馏进模型”。

19. 可复用 Know-how：如何训练自己的 Deep Research Agent

如果把 Tongyi 的做法压缩成可执行路线，可以得到下面的工程顺序：

8. 先定义 action space 与可验证任务。没有稳定工具和 outcome verifier，不要先上 RL。
9. 构建 synthetic QA engine：优先能控制 difficulty、multi-hop dependency、答案可验证的数据结构。
10. 在静态/模拟环境里合成完整 trajectory：至少覆盖 planning、reasoning、decision、tool call、observation。
11. 做 Agentic CPT：先短后长 context，混入少量 general pretraining data 防止能力偏科。
12. SFT cold start：对成功轨迹做 rejection sampling，优先保证输出格式、工具协议、停止行为可靠。
13. RL 前先做 environment hardening：cache、retry、QPS、fallback、tool telemetry。
14. 严格区分失败：policy failure、tool failure、timeout、context overflow、format error。只有 policy failure 才应该成为纯负奖励。
15. 使用可验证 0/1 reward 起步；若要加 process reward，先保证不会把工具噪声当模型错误。
16. 动态维护“中等难度问题池”，让 curriculum 随 policy 进步。
17. 长程任务不要只扩 context：训练/实现 summary、workspace reconstruction、external memory。

◆ 一个现实的优先级：Data Engine → Environment Reliability → SFT Baseline → Eval → RL。把 RL 放在环境和评测之前，通常只会得到更昂贵、更难诊断的失败。

20. 复现、部署、局限与建议

官方仓库提供 React inference 与 benchmark evaluation scripts。README 推荐 Python 3.10；需要配置 Serper (Web/Scholar)、Jina (页面阅读)、OpenAI-compatible summarization endpoint、DashScope file parser、SandboxFusion Python sandbox 和 model path 等。输入支持 JSON/JSONL，评测字段为 question/answer。[S2]

复现步骤	建议
准备模型	本地部署 Tongyi-DeepResearch-30B-A3B；模型卡支持 Transformers/vLLM 等常见方式
工具密钥	先只启用 Search+Visit，稳定后再加 Scholar/Python/File
运行 ReAct	使用 run_react_infer.sh / react_agent.py
日志	保存完整 messages、tool args、tool responses、termination reason
评测	严格使用 benchmark official evaluator；固定时间预算与 tool budget

成本	真实 Web API 前先用缓存或 simulation 做开发回归
----	------------------------------------

官方限制包括：128K 对最复杂 long-horizon task 仍不足；尚未证明该训练 pipeline 在显著更大 foundation model 上的可扩展性；希望进一步提升报告 fidelity / preference alignment；RL 效率仍可通过 partial rollout 等方向改进，但会引入 off-policy distribution shift。[S1][S3]

另外，开源仓库使用了替代工具实现以复现实验；技术报告附注说明内部系统存在一些内部 API 与 fallback 策略，公开版本提供替代实现，并声称通过测试可较好复现结果。[S1]

21. 关键结论与参考来源

如果只记住 Tongyi DeepResearch 的 8 个结论：

- Agent 能力可以像语言能力一样在 pre/post-training 之间增加一个 Agentic Mid-training 阶段。
- Deep Research 的瓶颈首先是数据和环境，而不是“换一个更复杂的 Agent Graph”。
- Synthetic data 的价值在于可规模化、可控难度、可验证，以及能够形成 policy-data flywheel。
- 训练环境需要分层：Prior World 负责规模，Simulation 负责迭代，Real World 负责最终 fidelity。
- SFT 负责稳定与格式，RL 负责探索与策略；二者职责不同。
- Agentic RL 的成功高度依赖工具环境稳定性，API 噪声会直接污染 reward。
- Long-horizon scaling 不仅是 context length，也包括 interaction turns、context reconstruction 和 parallel research paths。
- 未来 Deep Research 会同时存在两条路线：更强 Harness，以及更原生的 Agentic Model；工程系统会在两者之间重新分配复杂度。

参考来源（均为官方/一手资料）

[S1] Tongyi DeepResearch Technical Report, arXiv:2510.24701 v3 (2026-05-18)
<https://arxiv.org/pdf/2510.24701>

[S2] Alibaba-NLP/DeepResearch 官方 GitHub 仓库
<https://github.com/Alibaba-NLP/DeepResearch>

[S3] Tongyi DeepResearch 官方技术博客：A New Era of Open-Source AI Researchers
<https://tongyi-agent.github.io/blog/introducing-tongyi-deep-research/>

[S4] Hugging Face 官方模型卡：Alibaba-NLP/Tongyi-DeepResearch-30B-A3B
<https://huggingface.co/Alibaba-NLP/Tongyi-DeepResearch-30B-A3B>

[S5] AgentFounder / Scaling Agents via Continual Pre-training
<https://github.com/Alibaba-NLP/DeepResearch/tree/main/Agent/AgentFounder>

[S6] AgentScaler / Towards General Agentic Intelligence via Environment Scaling
<https://github.com/Alibaba-NLP/DeepResearch/tree/main/Agent/AgentScaler>

[S7] 官方 ReAct inference implementation
https://github.com/Alibaba-NLP/DeepResearch/blob/main/inference/react_agent.py

[S8] 官方 evaluation scripts
<https://github.com/Alibaba-NLP/DeepResearch/tree/main/evaluation>

研究说明

本报告中的具体训练流程、模型参数、工具集、实验分数与限制，以官方技术报告/仓库/模型卡为依据；对“为什么这样设计”“与其他系统的区别”“如何迁移到自建系统”的内容属于工程归纳与推断。商业模型对比仅用于理解论文当时的相对位置，不作为 2026-08-13 的实时排行榜。